

제 14 주:

사전 (Dictionary)



강 지 훈

jhkang@cnu.ac.kr

충남대학교 컴퓨터공학과



[제 14 주]

사전 (Dictionary)

□ 실습 목표

■ 이론적 관점

- 사전의 이해
- 추상 자료형의 개념
- 이진 검색 트리의 이해
- Call Back 의 개념

■ 구현적 관점

- 사전을 구현하는 다양한 방법
 - ◆ 특별히, 이진검색 트리의 구현
- 성능 측정 실험의 설계
- Call Back 의 구현

[문제 14-1] 사전의 성능 측정

□ 사전이란?

■ 사전이란?

- (단어, 설명)의 쌍들을 모아놓은 것
- 단어가 주어지면 그 단어의 설명을 찾는다.

■ 또 다른 예는?

- (학번, 평점)

■ 일반화 하면?

- (key, value)의 쌍을 모아 놓은 집합
- Key 값이 주어지면, 그에 해당하는 value 를 찾는다.

□ [프로그램 14-1]

- 사전을 3 가지로 구현하여 성능을 측정한다.
- 구현 방법의 종류
 - Sorted Array (Increasing order)
 - Sorted Linked Chain (Increasing order)
 - Binary Search Tree
- 성능 측정 기능
 - 삽입
 - 검색
 - 삭제

□ 입출력

■ 입력:

- 없음
- 필요한 데이터는 프로그램에서 생성

■ 출력: 성능 측정 결과:

- ◆ 데이터 크기 변화에 따른 성능 측정 결과

□ 성능 실험 데이터와 측정

■ 데이터 크기

- 10000, 20000, 30000, 40000, 50000

■ 데이터의 생성

- Random number를 생성하여 사용한다.

■ 측정

- 측정시간 단위: Micro (10^{-6}) second
- 각 행위 별로 측정한다.
 - ◆ 단위 행위의 함수 call 시간을 매번 측정하여 누적시킨다.

이 과제에서 필요한 객체는?

■ Controller 와 View:

- Class `AppController`
- Class `AppView`

■ Model:

● Dictionary 관련:

- ◆ Class `DictionaryElement<K,O>`
- ◆ Abstract class `Dictionary<K,O>`
- ◆ Class `DictionaryBySortedArray<K,O>` extends `Dictionary<K,O>`
- ◆ Class `DictionaryBySortedLinkedList<K,O>` extends `Dictionary<K,O>`
- ◆ Class `LinkedList<T>`
- ◆ Class `DictionaryByBinarySearchTree<K,O>` extends `Dictionary<K,O>`
- ◆ Class `BinaryNode<T>`

● 성능 측정 관련:

- ◆ Class `PerformanceMeasurement`
- ◆ Class `ParameterSet`
- ◆ Class `DataGenerator`
- ◆ Class `ExperimentResult`

□ 출력의 예[1] : 오름차순 데이터

■ SortedArray:

- 검색이 작은 이유?
- 삽입이 작은 이유?
- 삭제는 왜 큰지?
- 삭제가 다른 두 구현의 삽입에 비해 작은 이유?

■ SortedLinkedList:

- 삽입과 검색이 비슷한 이유?
- 삭제는 왜 작은지?
- 삽입이 SortedArray의 삭제보다 꽤 큰 이유? (7 배 정도?)

■ BinarySearchTree:

- 삽입과 검색이 비슷한 이유?
- 삭제는 왜 작은지?
- 결과가 SortedLinkedList와 유사하게 얻어지는 이유?

<<"Dictionary"의 성능 측정을 시작합니다.>>

<오름차순 데이터를 사용한 측정 (단위: micro second) >

"DictionaryBySortedArray"로 구현된 사전의 성능:

크기	삽입	검색	삭제
[10000]	737	822	29516
[20000]	1633	1859	117180
[30000]	2612	2632	262688
[40000]	3187	3464	466521
[50000]	4054	4362	723387

"DictionaryBySortedLinkedList"로 구현된 사전의 성능:

크기	삽입	검색	삭제
[10000]	188907	178717	339
[20000]	773859	766459	735
[30000]	1802691	1699382	1028
[40000]	3083175	3089925	1373
[50000]	4762675	4866273	1839

"DictionaryByBinarySearchTree"로 구현된 사전의 성능:

크기	삽입	검색	삭제
[10000]	189605	202948	473
[20000]	812322	796429	782
[30000]	1781055	1762547	1171
[40000]	3193179	3295570	1444
[50000]	4969473	4873252	2192

□ 출력의 예[2] : 내림차순 데이터

■ SortedArray:

- 삽입이 큰 이유?
- 삭제가 작은 이유?
- 삽입이 다른 두 구현의 삭제에 비해 작은 이유?

■ SortedLinkedList:

- 삽입과 검색이 비슷한 이유?
- 삭제는 왜 작은지?

■ BinarySearchTree:

- 삽입과 검색이 비슷한 이유?
- 삭제는 왜 작은지?
- 삽입이 SortedLinkedList의 삭제와 유사하게 얻어지는 이유?
- 검색이 SortedLinkedList의 검색과 유사하게 얻어지는 이유?

<내림차순 데이터를 사용한 측정 (단위: micro second) >

"DictionaryBySortedArray"로 구현된 사전의 성능:

크기	삽입	검색	삭제
[10000]	33742	827	626
[20000]	133760	1730	1287
[30000]	300578	2688	2007
[40000]	533484	3573	2699
[50000]	830285	4526	3407

"DictionaryBySortedLinkedList"로 구현된 사전의 성능:

크기	삽입	검색	삭제
[10000]	425	195359	189233
[20000]	806	554450	576074
[30000]	1203	1678390	1693241
[40000]	1598	2967987	2954214
[50000]	1999	4704652	4763727

"DictionaryByBinarySearchTree"로 구현된 사전의 성능:

크기	삽입	검색	삭제
[10000]	191083	190243	362
[20000]	774603	757117	731
[30000]	1767687	1736297	1100
[40000]	3178985	3142855	1507
[50000]	4927933	4845545	1833

□ 출력의 예[3] : 무작위 데이터

■ SortedArray:

- 삽입과 삭제가 비슷한 이유?
- 삭제가 다른 두 구현의 삽입에 비해 작은 이유?

■ SortedLinkedList:

- 검색이 삽입이나 삭제의 2 배 정도인 이유?
- 삽입과 삭제가 비슷한 이유?
- SortedArray에 비해 꽤 큰 이유?

■ BinarySearchTree:

- 삽입/삭제/검색이 비슷한 이유?
- 다른 구현에 비해 많이 작은 이유?

<무작위 데이터를 사용한 측정 (단위: micro second) >

"DictionaryBySortedArray"로 구현된 사전의 성능:

크기	삽입	검색	삭제
[10000]	17735	1500	15953
[20000]	71809	3782	65901
[30000]	151712	5950	138601
[40000]	273965	8215	252849
[50000]	429041	12181	378926

"DictionaryBySortedLinkedList"로 구현된 사전의 성능:

크기	삽입	검색	삭제
[10000]	225273	544206	199754
[20000]	1144517	2437445	1005288
[30000]	2668974	5500660	2424451
[40000]	4941131	9826466	4452542
[50000]	7862164	17914391	8215712

"DictionaryByBinarySearchTree"로 구현된 사전의 성능:

크기	삽입	검색	삭제
[10000]	1619	1640	1569
[20000]	3917	3647	3959
[30000]	6650	6331	6575
[40000]	9537	9018	9257
[50000]	12739	11778	12490

<<"Dictionary"의 성능 측정을 종료합니다.>>

main()

□ main()을 위한 class

```
public class DS1_14_학번_이름 {  
    public static void main(String[] args) {  
        // TODO Auto-generated method stub  
        AppController appController = new AppController();  
        appController.run ();  
    }  
}
```

Enum “ListOrder”



□ ListOrder

- 성능 측정 실험에 사용할 데이터 리스트의 3 가지 유형을 enum type 으로 정의한다:
 - Ascending / Descending / Random
- Java 의 열거 자료형 (enumeration data type 또는 간단히 enum type):
 - Enum type 만들기:
 - ◆ Enum type 으로 선언되는 변수가 가질 수 있는 모든 객체들을 identifier 형태로 열거한다.
 - ◆ Enum type 에 열거된 객체들은 순서 리스트 (ordered list) 가 된다.
 - 순서에 기반을 둔 멤버 함수들: ordinal(), compareTo()
 - Enum 도 class 이며, 단지 특수한 성격일 뿐:
 - ◆ enum 변수는 열거 자료형 객체 인스턴스를 소유한다.
 - ◆ Method 를 가질 수 있다:
 - static (class method) / non-static (instance method)
 - public / private
 - ◆ 당연히, enum 변수는 null 값을 가질 수 있다.
 - (예) ListOrder order = null;

□ ListOrder:

```
public enum ListOrder
```

```
{
```

```
    Ascending,  
    Descending,  
    Random ;
```

Enum 객체 이름들을 열거한다. 열거 순서는 의미를 갖는다.

```
    public static final String[] STRINGS_IN_KOREAN = new String[] {"오름차순", "내림차순", "무작위"} ;
```

```
    public String toStringInKorean () {  
        return ListOrder.STRINGS_IN_KOREAN [this.ordinal()] ;  
    }
```

```
} // End of enum "ListOrder"
```

ordinal()

객체가 enum 안에 선언되어 있는 순서를 얻는다.

맨 처음의 enum 객체는 그 순서 값 즉 ordinal() 값이 0 이며, 그 이후의 객체는 차례로 1 씩 증가한 순서 값을 갖는다.

Public instance method.

Enum 객체마다 한글로 된 문자열 이름을 부여하려는 목적의 함수이다.

(원래 enum 값으로 정의된 형태 그대로의 문자열을 얻으려면 "toString()" 함수를 사용하면 된다.)

현재 enum 객체의 순서 (this.ordinal()) 를 얻은 다음, 그 값을 STRINGS_IN_KOREAN[] 배열의 index 로 사용하여, 최종적으로 각 enum 객체의 한글 문자열 이름을 얻는다.

Class “AppController”

□ ApplicationController: 인스턴스 변수, 상수

```
public class ApplicationController {

    // 성능 측정 설정에 사용되는 상수
    private static final int APP_NUMBER_OF_EXPERIMENTS = 5 ;
    private static final int APP_FIRST_DATA_SIZE = 10000; // 측정 데이터 초기 크기
    private static final int APP_SIZE_INCREMENT = 10000; // 측정 데이터 크기 증가량

    // 비공개 변수들
    private AppView _appView ;
    private PerformanceMeasurement _measurement ;

    // 생성자
    public ApplicationController()
    {
        this._appView = new AppView() ;
    }

    // 비공개 함수의 구현
    .....

    // 공개 함수의 구현
    .....
} // End of class "AppController"
```

□ ApplicationController: run()

```
public void run() {
    this._appView.outputLine ("<<₩"Dictionary₩"의 성능 측정을 시작합니다.>>>₩n");
    this._measurement =
        new PerformanceMeasurement (
            APP_NUMBER_OF_EXPERIMENTS,
            APP_FIRST_DATA_SIZE,
            APP_SIZE_INCREMENT) ;
    this.showExperimentByListOrderType(ListOrder.Ascending) ;
    this.showExperimentByListOrderType(ListOrder.Descending) ;
    this.showExperimentByListOrderType(ListOrder.Random) ;
    this._appView.outputLine ("<<₩"Dictionary₩"의 성능 측정을 종료합니다.>>>₩n") ;
}
```

성능 측정을 수행할 객체를 생성

리스트의 주어진 순서 유형에 대해
성능 측정한 결과를 보여준다.

□ ApplicationController: showExperimentByListOrderType()

```
private void showExperimentByListOrderType (ListOrder anOrder) {
    this._appView.outputLine("<" + anOrder.toStringInKorean() +
        " 데이터를 사용한 측정 (단위: micro second) >");

    this.showExperimentByDictionaryAndListOrderType (
        new DictionaryBySortedArray<Integer, String>(
            this._measurement.parameterSet().maxDataSize(),
            anOrder);
    this._appView.outputLine("");

    this.showExperimentByDictionaryAndListOrderType (
        new DictionaryBySortedLinkedList<Integer, String>(),
        anOrder);
    this._appView.outputLine("");

    this.showExperimentByDictionaryAndListOrderType (
        new DictionaryByBinarySearchTree<Integer, String>(),
        anOrder);
    this._appView.outputLine("");
}
```

성능 측정에 사용된 사전의 종류와 리스트의 유형에 대한 측정 결과를 보여준다. 각 측정마다 필요한 사전 객체를 생성하여 보내고 있다.

<내림차순 데이터를 사용한 측정 (단위: mili second) >

"DictionaryBySortedArray"로 구현된 사전의 성능:

크기	삽입	검색	삭제
10000	32514	760	569
20000	132609	1601	1192
30000	292742	2513	1840
40000	512736	3305	2496
50000	800985	4205	3168

"DictionaryBySortedLinkedList"로 구현된 사전의 성능:

크기	삽입	검색	삭제
10000	410	187255	209335
20000	820	590586	658844
30000	1208	1906830	1842993
40000	1592	3123470	3076136
50000	2057	4879225	5000196

"DictionaryByBinaryTree"로 구현된 사전의 성능:

크기	삽입	검색	삭제
10000	200398	209614	356
20000	813702	845654	739
30000	1910062	1757572	1080
40000	2957996	3086052	1440
50000	5261691	4634717	1738

□ ApplicationController: showExperimentByDictionaryAndListOrderType() [1]

```
private void showExperimentByDictionaryAndListOrderType (
    Dictionary<Integer,String> aDictionary, ListOrder anOrder)
{
    this._appView.output("₩" + aDictionary.getClass().getName() + "₩로 구현된 사전의 성능:");
    this._appView.output(String.format("%6s", "크기 "));
    this._appView.output(String.format("%11s", "삽입"));
    this._appView.output(String.format("%11s", "검색"));
    this._appView.outputLine(String.format("%11s", "삭제"));

    UnitExperimentResult[] results =
        this._measurement.experimentByDictionaryAndListOrderType(aDictionary, anOrder);
    for ( int iteration = 0 ;
          iteration < this._measurement.parameterSet().numberOfExperiments() ;
          iteration++ )
    {
        this.showUnitExperimentResult(results[iteration]);
    }
}
```

중요:

이 함수를 call 하는 곳에서는 실제 측정에 사용할 사전 객체를 보내고 있다. 그러나 받는 쪽인 이 곳에서는 class type이 실제로 보낸 것의 class type으로 받는 것이 아니라, 상위 class로서 추상 자료형인 "Dictionary"로 받고 있다. 이유는?

"DictionaryBySortedArray"로 구현된 사전의 성능:

크기	삽입	검색	삭제
10000	32514	760	569
20000	132609	1601	1192
30000	292742	2513	1840
40000	512736	3305	2496
50000	800985	4205	3168

□ ApplicationController: showExperimentByDictionaryAndListOrderType() [2]

```
private void showExperimentByDictionaryAndListOrderType (
    Dictionary <Integer,String> aDictionary, ListOrder anOrder)
{
    this._appView.output("₩" + aDictionary.getClass().getName() + "₩로 구현된 사전의 성능:");
    this._appView.output(String.format("%6s", "크기 "));
    this._appView.output(String.format("%11s", "삽입"));
    this._appView.output(String.format("%11s", "검색"));
    this._appView.outputLine(String.format("%11s", "삭제"));

    UnitExperimentResult[] results =
        this._measurement.experimentByDictionaryAndListOrderType(aDictionary, anOrder);
    for ( int iteration = 0 ;
        iteration < this._measurement.parameterSet().numberOfExperiments() ;
        iteration++ )
    {
        this.showUnitExperimentResult(results[iteration]);
    }
}
```

주어진 사전과 리스트 유형에 대한 측정 결과를 한꺼번에 얻는다. 결과는 반복 회차 별 (즉 크기 별)로 배열에 저장되어 받는다.

특정 반복 회차 (iteration) 에 대한 단위 실험 결과를 출력한다.

"DictionaryBySortedArray"로 구현된 사전의 성능:

크기	삽입	검색	삭제
10000	32514	760	569
20000	132609	1601	1192
30000	292742	2513	1840
40000	512736	3305	2496
50000	800985	4205	3168

□ ApplicationController: showUnitExperimentResult()

```
private void showUnitExperimentResult (UnitExperimentResult aResult)
{
    this._appView.output(String.format("[%5d]", aResult.experimentSize())); ;
    this._appView.output(String.format("%12d", aResult.timeForAdd()/ 1000)); ;
    this._appView.output(String.format("%12d", aResult.timeForSearch()/ 1000)); ;
    this._appView.output(String.format("%12d", aResult.timeForRemove()/ 1000)); ;
}
```

측정 결과의 단위는 nano second.
결과를 출력하는 단위는 micro second 이므로 1000 으로 나누어 준다.

"DictionaryBySortedArray"로 구현된 사전의 성능:

크기	삽입	검색	삭제
10000	32514	760	569
20000	132609	1601	1192
30000	292742	2513	1840
40000	512736	3305	2496
50000	800985	4205	3168

Class “AppView”

□ AppView: 인스턴스 변수, 생성자

```
import java.util.Scanner;
```

```
public class AppView {  
    // 비공개 상수/변수들  
    private Scanner _scanner ;
```

```
    // 생성자
```

```
    public AppView ()  
    {  
        this._scanner = new Scanner(System.in) ;  
    }
```

```
    // 공개함수의 구현
```

```
    .....
```

```
}
```



□ Class "AppView" 의 공개함수

■ 출력을 위한 공개 함수

- public void output (String aString) {...}
- public void outputLine (String aString) {...}

Class “ParameterSet”

□ ParameterSet:

- 성능 측정에 필요한 매개변수 값들을 모아서 하나의 객체로 관리한다.
- 단위 실험 (Unit Experiment):
 - 이번 실험에서는 주어진 크기의 데이터를 사용하여 3 개의 행위 (삽입, 검색, 삭제) 의 성능을 측정한 값을 얻는 실험을 "단위 실험"으로 본다.
- 필요한 매개변수: 데이터의 크기를 일정하게 증가시키면서 " 단위 실험" 을 반복한다.
 - 측정 횟수 (Number Of Experiments): 데이터를 일정한 크기로 증가시키면서 "단위 실험"을 반복하는 횟수.
 - 첫 데이터 크기 (First Data Size): "단위 실험"의 초기 데이터 크기.
 - 크기 증가 (Size Increment): "단위 실험"을 반복하면서 매번 증가시키는 데이터의 크기

❑ ParameterSet: Instance variables, Getter/Setter

```
public class ParameterSet {  
    // Private Instance Variables  
    private int _numberOfExperiments;  
    private int _firstDataSize;  
    private int _sizeIncrement;  
  
    // Getter/Setter  
    public int numberOfExperiments() {...}  
    public void setNumberOfExperiments (int newNumberOfExperiments) {...}  
  
    public int firstDataSize () {...}  
    public void setFirstDataSize (int newFirstDataSize) {...}  
  
    public int sizeIncrement () {...}  
    public void setSizeIncrement (int newSizeIncrement) {...}  
  
    // Instance variable 이 존재하지 않는 getter: 계산을 하여 얻는다  
    public int maxDataSize () {  
        return (this.firstDataSize() +  
                (this.sizeIncrement() * (this.numberOfExperiments()-1))) ;  
    }  
}
```

□ ParameterSet: 생성자

```
public class ParameterSet {  
    // Private Instance Variables  
    .....  
  
    // Getter/Setter  
    .....  
  
    // Constructors  
    public ParameterSet () { // 기본 생성자  
    }  
    public ParameterSet (  
        int givenNumberOfExperiments,  
        int givenFirstDataSize,  
        int givenSizeIncrement)  
    {  
        this.setNumberOfExperiments (givenNumberOfExperiments) ;  
        this.setFirstDataSize (givenFirstDataSize) ;  
        this.setSizeIncrement (givenSizeIncrement) ;  
    }  
} // End of class "ParameterSet"
```

Static Class “DataGenerator”

□ Static Class

- "Static" class is a class that has NEVER any object instance.
- But, JAVA does not support the declaration of a static class.
- Then, how can we declare practically a static class in JAVA?
 - We declare a class as "final" since any static class do not need to be inherited.
 - There should be no instance variable in the class.
 - We prevent from constructing any object by declaring any constructor as "private".
 - All the methods should be "static".

□ 데이터 리스트 생성

■ 조건:

- 중복된 값이 없게 한다.
- 주어진 크기 (aSize) 의 리스트를 생성한다.
- 세 가지 형태의 리스트를 생성한다.

■ 오름차순 (ascending) 리스트: (크기 aSize)

- 0, 1, 2, 3, ... , (aSize-1)

■ 내림차순 (descending) 리스트:

- (aSize-1), (aSize -2), ... , 3, 2, 1, 0

■ 무작위 (random) 리스트:

- 먼저, 오름차순 리스트를 생성한다.
- 리스트의 원소를 차례대로 무작위 위치의 원소와 맞바꾼다.
 - ◆ 무작위 위치는 무작위 수를 생성하여 결정한다.

❑ DataGenerator: public static methods [1]

```
import java.util.Random ;

public final class DataGenerator { // static class
    // Constructor
    private DataGenerator() { } ;

    // Public "static" methods:
    public static int[] ascendingList (int aSize) {
        if (aSize > 0) {
            int[] list = new int[aSize];
            for (int i = 0; i < aSize; i++) {
                list[i] = i ;
            }
            return list ;
        }
        return null;
    }
}
```

□ DataGenerator: public static methods [2]

```
public static int[] descendingList (int aSize) {
    if (aSize > 0) {
        int[] list = new int[aSize];
        for (int i = 0; i < aSize; i++) {
            list[i] = aSize - i - 1 ;
        }
        return list ;
    }
    return null;
}
```

```
public static int[] randomList (int aSize) {
    if (aSize > 0) {
        int[] list = new int[aSize] ;
        for (int i = 0; i < aSize; i++) {
            list[i] = i ;
        }
        Random random = new Random ();
        for (int i = 0; i < aSize; i++) {
            int j = random.nextInt(aSize) ;
            int temp = list[i];
            list[i] = list[j] ;
            list[j] = temp;
        }
        return list;
    }
    return null;
}
} // End of class "DataGenerator"
```

Class

"PerformanceMeasurement"

❑ PerformanceMeasurement: 상수, 비공개 인스턴스 변수

```
public class PerformanceMeasurement {  
    // Constants  
    private static final int DEFAULT_NUMBER_OF_EXPERIMENTS = 5 ;  
    private static final int DEFAULT_FIRST_DATA_SIZE = 10000 ;  
    private static final int DEFAULT_SIZE_INCREMENT = 10000 ;  
  
    // Instance Variables  
    private ParameterSet _parameterSet;  
  
    private int[] _ascendingList ;  
    private int[] _descendingList ;  
    private int[] _randomList ;  
}
```

□ PerformanceMeasurement: Getter/Setter

```
public class PerformanceMeasurement {  
    // Constants  
    .....  
  
    // Instance Variables  
    .....  
  
    // Getter/Setter  
    public ParameterSet parameterSet () {  
        return this._parameterSet ;  
    }  
    public void setParameterSet (ParameterSet newParameterSet) {  
        this._parameterSet = newParameterSet ;  
    }  
  
    public int[] ascendingList () {  
        return this._ascendingList ;  
    }  
    public int[] descendingList () {  
        return this._descendingList ;  
    }  
    public int[] randomList () {  
        return this._randomList ;  
    }  
}
```

PerformanceMeasurement: 사전 생성자

```
public class PerformanceMeasurement {  
    // Constants  
    .....  
  
    // Instance Variables  
    .....  
  
    // Getter/Setter  
    .....  
  
    // Constructors  
    public PerformanceMeasurement () {  
        this.setParameterSet (new ParameterSet (  
            PerformanceMeasurement.DEFAULT_NUMBER_OF_EXPERIMENTS,  
            PerformanceMeasurement.DEFAULT_FIRST_DATA_SIZE,  
            PerformanceMeasurement.DEFAULT_SIZE_INCREMENT));  
        this.generateData() ;  
    }  
    public PerformanceMeasurement (  
        int givenNumberOfExperiments,  
        int givenFirstDataSize,  
        int givenSizeIncrement)  
    {  
        this.setParameterSet (new ParameterSet (  
            givenNumberOfExperiments,  
            givenFirstDataSize,  
            givenSizeIncrement));  
        this.generateData() ;  
    }  
}
```


□ PerformanceMeasurement: generateData()

// 측정 실험에 사용할 3 가지 데이터 리스트들을
// 최대 크기로 생성해 놓는다.

```
public void generateData() {  
    this._ascendingList =  
        DataGenerator.ascendingList (this.parameterSet().maxDataSize()) ;  
    this._descendingList =  
        DataGenerator.descendingList (this.parameterSet().maxDataSize()) ;  
    this._randomList =  
        DataGenerator.randomList (this.parameterSet().maxDataSize()) ;  
}
```

❑ PerformanceMeasurement: experimentList()

// 주어진 "anOrder" 에 해당하는 데이터 리스트를 돌려준다.

```
public int[] experimentList (ListOrder anOrder) {  
    if (anOrder == ListOrder.Ascending) {  
        return this._ascendingList ;  
    }  
    else if (anOrder == ListOrder.Descending) {  
        return this._descendingList ;  
    }  
    else if (anOrder == ListOrder.Random) {  
        return this._randomList ;  
    }  
    else {  
        return null ;  
    }  
}
```

□ PerformanceMeasurement: unitExperiment()

// 단위 측정 실험을 실행한다. 그 측정 결과를 돌려준다.

```
private UnitExperimentResult unitExperiment (Dictionary<Integer, String> aDictionary, ListOrder anOrder, int dataSize) {
    long startTime ;
    long stopTime ;
    int[] experimentList = this.experimentList (anOrder) ;

    long timeForAdd = 0 ;
    for (int i = 0; i < dataSize; i++) {
        startTime = System.nanoTime() ;
        aDictionary.addKeyAndObject (experimentList[i], null) ;
        stopTime = System.nanoTime() ;
        timeForAdd += (stopTime - startTime) ;
    }

    long timeForSearch = 0 ;
    for (int i = 0; i < dataSize; i++) {
        startTime = System.nanoTime() ;
        aDictionary.objectForKey (experimentList[i]) ;
        stopTime = System.nanoTime() ;
        timeForSearch += (stopTime - startTime) ;
    }

    long timeForRemove = 0 ;
    for (int i = 0; i < dataSize; i++) {
        ..... // 여기를 채우시오. 사용할 함수: aDictionary.removeObjectForKey()
    }

    return (new UnitExperimentResult (dataSize, timeForAdd, timeForSearch, timeForRemove)) ;

} // End of "unitExperiment()"
```

❏ PerformanceMeasurement: experimentByDictionaryAndListOrderType()

// 주어진 사전과 리스트 타입의 데이터 리스트를 사용하여 크기 별로 단위 측정 실험을 실행한다.
 // 그 측정 결과를 돌려준다.

```
public UnitExperimentResult[] experimentByDictionaryAndListOrderType (
    Dictionary<Integer, String> aDictionary, ListOrder anOrder)
{
    UnitExperimentResult[] experimentResults =
        new UnitExperimentResult [this.parameterSet().numberOfExperiments()];

    // 측정 결과를 안정시키기 위해서 측정과 무관하게 미리 실행한다.
    this.unitExperiment (aDictionary, anOrder, this.parameterSet().maxDataSize());
    System.gc(); // 측정 전에 미리 garbage collection 을 강제로 실시하여,
                // 측정을 실행하는 동안에 garbage collection이 되도록 발생하지 않게 함으로써
                // 가능한 한 정확한 측정 값이 얻어지도록 한다.

    int dataSize = this.parameterSet().firstDataSize();
    for (int iteration = 0 ; iteration < this.parameterSet().numberOfExperiments() ; iteration++) {
        aDictionary.clear();
        experimentResults[iteration] = this.unitExperiment (aDictionary, anOrder, dataSize);
        dataSize += this.parameterSet().sizeIncrement();
    }
    return experimentResults ;
}

} // End of "experimentByDictionaryAndListOrderType()"
```

Class “UnitExperimentResult”

UnitExperimentResult

```
public class UnitExperimentResult {  
    private int    _experimentSize ;  
    private long   _timeForAdd ;  
    private long   _timeForSearch ;  
    private long   _timeForRemove ;  
}
```

□ UnitExperimentResult: 생성자

- public UnitExperimentResult()
- public UnitExperimentResult(
 int givenExperimentSize,
 long givenTimeForAdd,
 long givenTimeForSearch,
 long givenTimeForRemove)

UnitExperimentResult: Getter/Setter

- `public int experimentSize()`
- `public void setExperimentSize(int newExperimentSize)`

- `public double timeForAdd()`
- `public void setTimeForAdd(double newTimeForAdd)`

- `public double timeForSearch()`
- `public void setTimeForSearch (double newTimeForSearch)`

- `public double timeForRemove()`
- `public void setTimeForRemove (double newTimeForRemove)`

Class “DictionaryElement”

Class DictionaryElement

```
public class DictionaryElement<Key, Obj> {  
    private Key _key ;  
    private Obj _object ;  
}
```

□ DictionaryElement: 생성자, getter/setter

■ 생성자:

- public DictionaryElement ()
- public DictionaryElement (Key givenKey, Obj givenObject)

- public Key key ()
- public void setKey (Key newKey)

- public Obj object ()
- public void setObject (Obj newObject)

Class “LinkedList”

□ ListNode

```
public class ListNode<E> {  
    // Instance variables  
    private E          _element ;  
    private ListNode<E> _next ;  
  
    // Getter/Setter  
    public E element () {...}  
    public void setElement (E newElement) {...}  
  
    public ListNode<E> next() {...}  
    public void setNext (ListNode<E> newNext) {...}  
  
    // Constructors  
    public ListNode() {...}  
    public ListNode (E givenElement, ListNode<E> givenNext) {...}  
  
} // End of class "ListNode"
```

각 Dictionary 의 구현

□ Dictionary:

■ 3 가지 구현:

- Class "DictionaryBySortedLinkedList"
- Class "DictionaryBySortedArray"
- Class "DictionaryByBinarySearchTree"

■ 이 3 가지 class 를 추상화 한 상위 class 를 정의한다:

- Abstract class "Dictionary"
- 그러므로, 3 가지 구현 별 class 들은 각각 abstract class "Dictionary" 를 상속 받는다.

□ Dictionary:

- 측정 실험에 사용할 함수:
 - 삽입: `addKeyAndObject ()`
 - 검색: `objectForKey()`
 - 삭제: `removeObjectForKey()`

Abstract Class “Dictionary”

□ Dictionary: 공개함수

■ 추상 class 에 구현되는 함수:

- `public int size() {...} ;`
 - ◆ "size()" 는 모든 사전 class 에서 동일하게 구현되므로 추상 class "Dictionary"에 구현한다.
- `public boolean isEmpty() {...} ;`
 - ◆ "size()" 에 기반을 두고 구현하므로, 역시 모든 사전 class 에서 동일하게 구현된다. 따라서 추상 class "Dictionary"에 구현한다.

■ 추상 class 에 구현되지 않고, 하위 class에서 구현해야 할 추상 함수:

- `public abstract boolean isFull () ;`
- `public abstract boolean keyDoesExist (Key aKey) ;`
- `public abstract Obj objectForKey (Key aKey) ;`
- `public abstract boolean addKeyAndObject (Key aKey, Obj anObject) ;`
- `public abstract Obj removeObjectForKey (Key aKey) ;`
- `public abstract void clear () ;`

Dictionary:

```
public abstract class Dictionary<Key extends Comparable<Key>, Obj> {
    // Instance variable
    private int _size; // "private" 임에 유의. 상속 받는 class에서는 getter/setter를 통해서만 접근한다.

    // Getter/Setter
    public int size () {...}
    protected void setSize (int newSize) {...}
        // setSize()는 사전의 크기를 변경시킨다. 삽입과 삭제의 행위가 실행될 때 변경된다.
        // 따라서 "public" 함수가 아니어야 한다. 외부에 공개되면 안 된다.
        // "private"이 아니고 "protected"인 것은, 상속 받는 class에서는 사용할 수 있게 하기 위함이다.
        // 상속 받는 class에서는, 인스턴스 변수 "_size"를 직접 사용할 수 없게 설계하는 것이 적절한 방법이다.

    // Constructors
    public Dictionary () {
        this.setSize (0); // 상속받는 class의 생성자는 암묵적으로 상위 class의 생성자를 call한다는 것을 잊지 말 것!
    }

    // Public non-abstract method: 이 class에서 구현되어야 한다.
    public boolean isEmpty () {...}

    // Public abstract methods
    public abstract boolean isFull ();
    public abstract boolean keyDoesExist (Key aKey);
    public abstract Obj objectForKey (Key aKey);
    public abstract boolean addKeyAndObject (Key aKey, Obj anObject);
    public abstract Obj removeObjectForKey (Key aKey);
    public abstract void clear ();

} // End of class "LinkedList"
```

Class “DictionaryBySortedArray”

DictionaryBySortedArray

```
public class DictionaryBySortedArray<Key extends Comparable<Key>, Obj>
    extends Dictionary<Key,Obj>
{
    // Constants
    private static final int DEFAULT_CAPACITY = 100000 ;

    // Private instance variables
    private int _capacity ;
    private int _size; // 선언하지 않는 이유는?
    private DictionaryElement<Key, Obj>[] _elements ;

    // Getter/Setter
    public int capacity () {...}
    private void setCapacity (int newCapacity) {...}

    // Constructors
    public DictionaryBySortedArray () {
        this (DictionaryBySortedArray.DEFAULT_CAPACITY) ;
    }
    @SuppressWarnings("unchecked")
    public DictionaryBySortedArray (int givenCapacity) {
        this.setCapacity (givenCapacity);
        this._elements = new DictionaryElement[this.capacity()] ;
    }
}
```

□ DictionaryBySortedArray: private methods [1]

// Private methods

private int positionFor (Key aKey) { // **Binary search**

int left = 0 ;

int right = this.size() - 1 ;

while (left <= right) {

int mid =(left+right) / 2 ;

switch (aKey.compareTo (this._elements[mid].key())) {

case -1: // Continue to search the left part

right = mid -1 ;

break ;

case 0: // We have found an element with the given "aKey" at the position "mid".

return mid ; // We return the position as a POSITIVE value.

case +1: // Continue to search the right part.

left = mid + 1 ;

break ;

}

}

// At this point, we have failed to find an element with the given "aKey".

// So, we return the failed position as a NEGATIVE value.

//

// The failed position can become a zero (0).

// And, the successfully found position can also be zero.

// This two cases cannot be distinguished because the negative of zero is still zero.

// In order to avoid this situation,

// we add one (1) to a value of the failed position (actually "left") before taking negative value.

//

return -(left+1) ;

}

□ DictionaryBySortedArray: private methods [2]

```
private void makeRoomAt (int aPosition) {  
    for ( int i = this.size() ; i > aPosition ; i-- ) {  
        this._elements[i] = this._elements[i-1] ;  
    }  
}  
  
private void removeGapAt (int aPosition) {  
    for ( int i = aPosition ; i < this.size() -1 ; i++ ) {  
        this._elements[i] = this._elements[i+1] ;  
    }  
}
```

□ DictionaryBySortedArray: public methods [1]

// Public methods

```
@Override
public boolean isFull () {...}
```

```
@Override
public boolean keyDoesExist (Key aKey) {...} // "positionFor()" 를 사용하여 구현한다.
```

```
@Override
public boolean addKeyAndObject (Key aKey, Obj anObject) {
    int positionForAdd = this.positionFor (aKey) ;
    if ( positionForAdd >= 0 ) {
        return false; // "aKey" already exist.
    }
    // "positionForAdd" now has the failed position as a negative value.
    // But be careful ! We can get the correct position by taking the negative value
    // and then subtracting 1.
    //
    positionForAdd = - (positionForAdd + 1) ;
    this.makeRoomAt (positionForAdd) ;
    this._elements[positionForAdd] = new DictionaryElement<Key,Obj>(aKey,anObject) ;
    this.setSize (this.size() + 1) ;
    return true ;
}
```


□ DictionaryBySortedArray: public methods [2]

```
@Override
public Obj objectForKey (Key aKey) {
    int positionWithKey = this.positionFor (aKey) ;
    if ( positionWithKey < 0 ) {
        return null ;
    }
    return this._elements[positionWithKey].object() ;
}
```

```
@Override
public Obj removeObjectForKey (Key aKey) {
    int positionForRemove = this.positionFor (aKey) ;
    if ( positionForRemove < 0 ) {
        return null ;
    }
    Obj removedObject = this._elements[positionForRemove].object() ;
    this.removeGapAt (positionForRemove) ;
    this.setSize (this.size() -1) ;
    return removedObject ;
}
```

```
@Override
public void clear () {
    this.setSize(0) ;
}
```

Class

“DictionaryBySortedLinkedList”

DictionaryBySortedLinkedList

```
public class DictionaryBySortedLinkedList<Key extends Comparable<Key>, Obj>
    extends Dictionary<Key,Obj>
{
    // Private instance variables
    private ListNode<DictionaryElement<Key,Obj>> _head ;
    private int _size; // 선언하지 않는 이유는? (상속 관계를 볼 것)

    // Getter/Setter
    private ListNode<DictionaryElement<Key,Obj>> head () {...}
    private void setHead (ListNode<DictionaryElement<Key,Obj>> newHead) {...}

    // Constructors
    public DictionaryBySortedLinkedList () {
        this.clear () ;
    }
}
```

□ DictionaryBySortedList: public methods[1]

```
// Public methods
```

```
@Override
```

```
public boolean isFull () {
    return false ;
}
```

```
@Override
```

```
public boolean keyDoesExist (Key aKey) {
    ListNode<DictionaryElement<Key,Obj>> current = this.head() ;
    while ( current != null ) {
        switch ( current.element().key().compareTo(aKey) ) {
            case -1:
                current = current.next() ;
                break ;
            case 0:
                return true; // Found
            case +1:
                return false; // Not found
        }
    }
    return false;
}
```

□ DictionaryBySortedLinkedList: public methods[2]

@Override

```
public Obj objectForKey (Key aKey) {
    ListNode<DictionaryElement<Key,Obj>> current = this.head() ;
    while ( (current != null) && (current.element().key().compareTo(aKey) < 0) ) {
        current = current.next() ;
    }
    if ( (current != null) && (current.element().key().compareTo(aKey) == 0 ) ) {
        // Found
        return current.element().object() ; // Found
    }
    else {
        return null ; // Not found
    }
}
```

□ DictionaryBySortedLinkedList: public methods[3]

@Override

```
public boolean addKeyAndObject (Key aKey, Obj anObject) {
    ListNode<DictionaryElement<Key,Obj>> previous = null ;
    ListNode<DictionaryElement<Key,Obj>> current = this.head() ;
    while ( (current != null) && (current.element().key().compareTo(aKey) < 0) ) {
        previous = current ;
        current = current.next() ;
    }
    if ( (current != null) && (current.element().key().compareTo(aKey) == 0) ) {
        return false; // "aKey" already exists.
    }
    DictionaryElement<Key,Obj> addedElement =
        new DictionaryElement<Key,Obj>(aKey, anObject) ;
    ListNode<DictionaryElement<Key,Obj>> addedNode =
        new ListNode<DictionaryElement<Key,Obj>>() ;
    addedNode.setElement (addedElement) ;
    addedNode.setNext (current) ;
    if ( previous == null ) { // 리스트의 맨 앞에 삽입해야 한다
        this.setHead (addedNode) ;
    }
    else {
        previous.setNext(addedNode) ;
    }
    this.setSize (this.size()+1);
    return true; // Adding has been succeeded.
} // End of "addKeyAndObject()"
```

□ DictionaryBySortedLinkedList: public methods[4]

```

@Override
public Obj removeObjectForKey (Key aKey) {
    ListNode<DictionaryElement<Key,Obj>> previous = null ;
    ListNode<DictionaryElement<Key,Obj>> current = this.head() ;
    while ( (current != null) && (current.element().key().compareTo(aKey) < 0) ) {
        previous = current ;
        current = current.next() ;
    }
    if ( (current != null) && (current.element().key().compareTo(aKey) == 0 ) {
        // Found
        if ( current == this.head() ) { // The node to be removed is the head node.
            this.setHead (current.next()) ;
        }
        else { // The node to be removed is not the head node.
            previous.setNext (current.next()) ;
        }
        this.setSize (this.size()-1);
        return current.element().object() ; // Found
    }
    else {
        return null ; // Not found
    }
}

@Override
public boolean clear () {
    this.setSize (0) ;
    this.setHead (null) ;
}

```

Class “BinaryNode”

BinaryNode

```

public class BinaryNode<E> {
    // Private instance variables
    private E          _element;
    private BinaryNode<E> _left ;
    private BinaryNode<E> _right ;

    // Getter/Setter
    public E element () {...}
    public void setElement (E newElement) {...}

    public BinaryNode<E> left () {...}
    public void setLeft (BinaryNode<E> newLeft) {...}

    public BinaryNode<E> right () {...}
    public void setRight (BinaryNode<E> newRight) {...}

    // Constructors
    public BinaryNode () {
        this.setElement (null) ;
        this.setLeft (null) ;
        this.setRight (null) ;
    }
    public BinaryNode (E givenElement, BinaryNode<E> givenLeft, BinaryNode<E> givenRight) {
        this.setElement (givenElement) ;
        this.setLeft (givenLeft) ;
        this.setRight (givenRight) ;
    }
} // End of class "BinaryNode"
  
```

Class “DictionaryByBinarySearchTree”

DictionaryByBinarySearchTree

```
public class DictionaryByBinarySearchTree<Key extends Comparable<Key>, Obj>
    extends Dictionary<Key,Obj>
{
    // Private instance variables
    private BinaryNode<DictionaryElement<Key,Obj>> _root ;
    private int _size ;

    // Getter/Setter
    private BinaryNode<DictionaryElement<Key,Obj>> root () {...}
    private void setRoot (BinaryNode<DictionaryElement<Key,Obj>> newRoot) {...}

    // Constructor
    public DictionaryByBinarySearchTree () {
        this.clear () ;
    }
}
```

DictionaryByBinarySearchTree: 공개함수[1]

```
// Public methods
```

```
@Override
```

```
public boolean isFull() {
    return false ; // Always false
}
```

```
private DictionaryElement<Key,Obj> elementForKey (Key aKey) {
    BinaryNode<DictionaryElement<Key,Obj>> current = this.root() ;
    while ( current != null ) {
        if ( current.element().key().compareTo(aKey) == 0 ) {
            return current.element() ;
        }
        else if ( current.element().key().compareTo(aKey) > 0 ) {
            current = current.left() ;
        }
        else {
            current = current.right() ;
        }
    }
    return null;
}
```

```
@Override
```

```
public boolean keyDoesExist (Key aKey) {
    return ( this.elementForKey (aKey) != null ) ;
}
```

```
@Override
```

```
public Obj objectForKey (Key aKey) {
    DictionaryElement<Key, Obj> element = this.elementForKey (aKey) ;
    if ( element != null ) {
        return element.object() ;
    }
    else {
        return null ;
    }
}
```

DictionaryByBinarySearchTree: 공개함수[2]

```

@Override
public boolean addKeyAndObject (Key aKey, Obj anObject) {
    DictionaryElement<Key,Obj> elementForAdd = new DictionaryElement<Key,Obj>(aKey, anObject) ;
    BinaryNode<DictionaryElement<Key,Obj>> nodeForAdd =
        new BinaryNode<DictionaryElement<Key,Obj>>(elementForAdd, null, null) ;

    if ( this.root() == null ) {
        this.setRoot (nodeForAdd) ;
        this.setSize (1);
        return true ;
    }
    BinaryNode<DictionaryElement<Key,Obj>> current = this.root() ;
    while ( aKey.compareTo(current.element().key()) != 0 ) {
        if ( aKey.compareTo (current.element().key()) < 0 ) {
            if ( current.left() == null ) {
                current.setLeft (nodeForAdd) ;
                this.setSize (this.size()+1) ;
                return true;
            }
            else {
                current = current.left() ;
            }
        }
        else {
            if ( current.right() == null ) {
                current.setRight (nodeForAdd) ;
                this.setSize (this.size()+1) ;
                return true;
            }
            else {
                current = current.right() ;
            }
        }
    }
    // End of while
    return false ;
} // End of "addKeyAndObject()"

```

DictionaryByBinarySearchTree: 공개함수[3]

```
private DictionaryElement<Key,Obj>
    removeRightMostElementOfLeftSubTree (BinaryNode<DictionaryElement<Key,Obj>> root)
{
    // At this point, "root" has non-empty left subtree.
    BinaryNode<DictionaryElement<Key,Obj>> leftOfRoot = root.left() ;
    if ( leftOfRoot.right() == null ) {
        root.setLeft (leftOfRoot.left()) ;
        return leftOfRoot.element() ;
    }
    else {
        // There exist at least one node in the right subtree of the left of the root.
        BinaryNode<DictionaryElement<Key,Obj>> parentOfRightMost = leftOfRoot ;
        BinaryNode<DictionaryElement<Key,Obj>> rightMost = parentOfRightMost.right() ;
        while ( rightMost.right() != null ) {
            parentOfRightMost = rightMost ;
            rightMost = rightMost.right() ;
        }
        // We have found the right-most node so that it is owned by "rightMost".
        parentOfRightMost.setRight (rightMost.left()) ;
        return rightMost.element() ;
    }
} // End of "removeRightMostElementOfLeftSubTree()"
```

DictionaryByBinarySearchTree: 공개함수[4]

@Override

```
public Obj removeObjectForKey (Key aKey) {  
    if ( this.root() == null ) {  
        return null ;  
    }  
    if ( aKey.compareTo(this.root().element().key()) == 0 ) {  
        Obj objectForRemove = this.root().element().object() ;  
        if ( (this.root().left() == null) && (this.root().right() == null) ) {  
            this.setRoot (null) ; // Root-only tree  
        }  
        else if ( this.root().left() == null ) {  
            this.setRoot (this.root().right()) ;  
        }  
        else if ( this.root().right() == null ) {  
            this.setRoot (this.root().left()) ;  
        }  
        else {  
            this.root().setElement (this.removeRightMostElementOfLeftSubTree(this.root()) ) ;  
        }  
        this.setSize (this.size()-1) ;  
        return objectForRemove;  
    }  
}
```

DictionaryByBinarySearchTree: 공개함수[5]

```

@Override
public Obj removeObjectForKey (Key aKey) {
    if ( this.root() == null ) {
        return null ;
    }
    if ( aKey.compareTo(this.root().element().key()) == 0 ) {
        .....
    }
    BinaryNode<DictionaryElement<Key,Obj>> current = this.root() ;
    BinaryNode<DictionaryElement<Key,Obj>> child = null ;
    do {
        if ( aKey.compareTo(current.element().key()) < 0 ) {
            child = current.left() ;
            if ( child == null ) {
                return null ;
            }
        }
        if ( aKey.compareTo(child.element().key()) == 0 ) {
            Obj objectForRemove = child.element().object() ;
            if ( child.left() == null && child.right() == null ) {
                current.setLeft(null) ;
            }
            else if ( child.left() == null ) {
                current.setLeft(child.right()) ;
            }
            else if ( child.right() == null ) {
                current.setLeft(child.left()) ;
            }
            else {
                child.setElement (this.removeRightMostElementOfLeftSubTree(child)) ;
            }
            this.setSize (this.size()-1) ;
            return objectForRemove;
        }
    }
}

```


DictionaryByBinarySearchTree: 공개함수[6]

@Override

public Obj removeObjectForKey (Key aKey) {

.....
BinaryNode<DictionaryElement<Key,Obj>> current = this.root() ;

BinaryNode<DictionaryElement<Key,Obj>> child = null ;

do {

if (aKey.compareTo(current.element().key()) < 0) {

.....
}

else {

child = current.right() ;

if (child == null) {

return null ;

}

if (aKey.compareTo(child.element().key()) == 0) {

Obj objectForRemove = child.element().object() ;

if (child.left() == null && child.right() == null) {

current.setRight(null) ;

}

else if (child.left() == null) {

current.setRight(child.right()) ;

}

else if (child.right() == null) {

current.setRight(child.left()) ;

}

else {

child.setElement (this.removeRightMostElementOfLeftSubTree(child)) ;

}

this.setSize (this.size()-1) ;

return objectForRemove;

}

}

current = child;

} while (true) ;

} // End of "removeObjectForKey()"

□ DictionaryByBinarySearchTree: 사전 공개함수[7]

```
@Override  
public void clear () {  
    this.setSize (0) ;  
    this.setRoot (null) ;  
} // End of "clear()"
```

요약

□ 측정 결과 분석 [1]: 오름차순 데이터

■ SortedArray:

- 검색이 작은 이유?
- 삽입이 작은 이유?
- 삭제는 왜 큰지?
- 삭제가 다른 두 구현의 삽입에 비해 작은 이유?

■ SortedLinkedList:

- 삽입과 검색이 비슷한 이유?
- 삭제는 왜 작은지?
- 삽입이 SortedArray의 삭제보다 꽤 큰 이유? (7 배 정도?)

■ BinarySearchTree:

- 삽입과 검색이 비슷한 이유?
- 삭제는 왜 작은지?
- 결과가 SortedLinkedList와 유사하게 얻어지는 이유?

<<"Dictionary"의 성능 측정을 시작합니다.>>

<오름차순 데이터를 사용한 측정 (단위: micro second) >

"DictionaryBySortedArray"로 구현된 사전의 성능:

크기	삽입	검색	삭제
[10000]	737	822	29516
[20000]	1633	1859	117180
[30000]	2612	2632	262688
[40000]	3187	3464	466521
[50000]	4054	4362	723387

"DictionaryBySortedLinkedList"로 구현된 사전의 성능:

크기	삽입	검색	삭제
[10000]	188907	178717	339
[20000]	773859	766459	735
[30000]	1802691	1699382	1028
[40000]	3083175	3089925	1373
[50000]	4762675	4866273	1839

"DictionaryByBinarySearchTree"로 구현된 사전의 성능:

크기	삽입	검색	삭제
[10000]	189605	202948	473
[20000]	812322	796429	782
[30000]	1781055	1762547	1171
[40000]	3193179	3295570	1444
[50000]	4969473	4873252	2192

□ 측정 결과 분석 [2]: 내림차순 데이터

■ SortedArray:

- 삽입이 큰 이유?
- 삭제가 작은 이유?
- 삽입이 다른 두 구현의 삭제에 비해 작은 이유?

■ SortedLinkedList:

- 삽입과 검색이 비슷한 이유?
- 삭제는 왜 작은지?

■ BinarySearchTree:

- 삽입과 검색이 비슷한 이유?
- 삭제는 왜 작은지?
- 삽입이 SortedLinkedList의 삭제와 유사하게 얻어지는 이유?
- 검색이 SortedLinkedList의 검색과 유사하게 얻어지는 이유?

<내림차순 데이터를 사용한 측정 (단위: micro second) >

"DictionaryBySortedArray"로 구현된 사전의 성능:

크기	삽입	검색	삭제
[10000]	33742	827	626
[20000]	133760	1730	1287
[30000]	300578	2688	2007
[40000]	533484	3573	2699
[50000]	830285	4526	3407

"DictionaryBySortedLinkedList"로 구현된 사전의 성능:

크기	삽입	검색	삭제
[10000]	425	195359	189233
[20000]	806	554450	576074
[30000]	1203	1678390	1693241
[40000]	1598	2967987	2954214
[50000]	1999	4704652	4763727

"DictionaryByBinarySearchTree"로 구현된 사전의 성능:

크기	삽입	검색	삭제
[10000]	191083	190243	362
[20000]	774603	757117	731
[30000]	1767687	1736297	1100
[40000]	3178985	3142855	1507
[50000]	4927933	4845545	1833

□ 측정 결과 분석 [3]: 무작위 데이터

■ SortedArray:

- 삽입과 삭제가 비슷한 이유?
- 삭제가 다른 두 구현의 삽입에 비해 작은 이유?

■ SortedLinkedList:

- 검색이 삽입이나 삭제의 2 배 정도인 이유?
- 삽입과 삭제가 비슷한 이유?
- SortedArray에 비해 꽤 큰 이유?

■ BinarySearchTree:

- 삽입/삭제/검색이 비슷한 이유?
- 다른 구현에 비해 많이 작은 이유?

<무작위 데이터를 사용한 측정 (단위: micro second) >

"DictionaryBySortedArray"로 구현된 사전의 성능:

크기	삽입	검색	삭제
[10000]	17735	1500	15953
[20000]	71809	3782	65901
[30000]	151712	5950	138601
[40000]	273965	8215	252849
[50000]	429041	12181	378926

"DictionaryBySortedLinkedList"로 구현된 사전의 성능:

크기	삽입	검색	삭제
[10000]	225273	544206	199754
[20000]	1144517	2437445	1005288
[30000]	2668974	5500660	2424451
[40000]	4941131	9826466	4452542
[50000]	7862164	17914391	8215712

"DictionaryByBinarySearchTree"로 구현된 사전의 성능:

크기	삽입	검색	삭제
[10000]	1619	1640	1569
[20000]	3917	3647	3959
[30000]	6650	6331	6575
[40000]	9537	9018	9257
[50000]	12739	11778	12490

<<"Dictionary"의 성능 측정을 종료합니다.>>

□ [문제 13-1] 요약

- 사전(Dictionary)의 개념과 그 구현에 따른 성능 차이를 이해한다.
 - 동일한 기능이 구현에 따라 어떻게 성능 차이가 나는지 이해한다.
 - 데이터의 크기의 증가에 따른 측정 시간의 변화가 이론적인 Big-Oh 를 반영하고 있는지 파악한다.
- 측정 결과 분석의 질문과 위에서 언급한 사항에 대해, 자신의 실험 결과를 이해하고, 각자의 분석을 논하시오.

[문제 14-2]

Binary Tree Traversals



□ [문제 14-2]

- Binary Tree의 Traversal의 실행을 구현하는 방법을 알아본다.
 - inorder traversal (중위 탐색) 을 구현한다.
- 이진트리의 모습을 출력할 방법을 알아본다.
 - 새로운 노드를 삽입할 때마다, 트리가 변하는 모습을 출력한다.
 - 노드를 하나씩 삭제할 때마다, 트리가 변하는 모습을 출력한다.
 - 이진 트리의 모습을 출력하는 것도, 개념적으로 트리 탐색의 문제이다.

□ 입력

- 키보드로부터의 입력은 받지 않는다.
- 데이터 크기를 달리하여, 동일한 작업을 2번 실행한다.
- 데이터 크기:
 - 첫 번째 데이터 크기 : 10
 - 두 번째 데이터 크기 : 20
- 사용할 데이터는 class "DataGenerator"를 사용하여 미리 만들어 놓는다.
 - 생성할 데이터 리스트는 무작위 데이터 리스트로 한다.

□ 사전을 사용하여 할 일

■ 삽입 단계:

- 미리 생성한 무작위 리스트의 값들을 이진검색트리로 구현한 사전 (class "DictionaryByBinarySearchTree")에 차례로 삽입한다.
- 리스트의 원소의 값을 Key 값으로 사용한다. (Integer)
- 삽입 순서를 Object 값으로 사용한다. (Integer)

■ 중위 탐색 단계:

- 삽입이 완료된 이진 트리를 중위 탐색 (inorder traverse) 한다.

■ 삭제 단계:

- 앞 단계에서 삽입이 완료된 사전의 상태를 그대로 사용한다.
- 미리 생성한 무작위 리스트의 값들을 key 값으로 갖는 <Key,Obj> 쌍을 사전에서 차례로 삭제한다.
- 당연히, 리스트의 원소의 값을 Key 값으로 사용한다.

□ 출력

■ 삽입 단계:

- 삽입할 때 마다, 삽입 직후에 트리의 모습을 출력한다.

■ 중위 탐색 단계:

- 이진 트리를 중위 탐색하여 그 순서대로 (Key, Object) 쌍을 출력한다.

■ 삭제 단계:

- 삭제할 때 마다, 삭제 직전에 트리의 모습을 출력한다.

출력 화면 [1] : 삽입 단계

<< 삽입 과정에서의 이진검색트리의 변화 >>

2(0) 원소를 삽입한 후의 이진검색트리:

2(0)

7(1) 원소를 삽입한 후의 이진검색트리:

7(1)

2(0)

4(2) 원소를 삽입한 후의 이진검색트리:

7(1)

4(2)

2(0)

6(3) 원소를 삽입한 후의 이진검색트리:

7(1)

6(3)

4(2)

2(0)

5(4) 원소를 삽입한 후의 이진검색트리:

7(1)

6(3)

5(4)

4(2)

2(0)

9(5) 원소를 삽입한 후의 이진검색트리:

9(5)

7(1)

6(3)

5(4)

4(2)

2(0)

3(6) 원소를 삽입한 후의 이진검색트리:

9(5)

7(1)

6(3)

5(4)

4(2)

3(6)

2(0)

0(7) 원소를 삽입한 후의 이진검색트리:

9(5)

7(1)

6(3)

5(4)

4(2)

3(6)

2(0)

0(7)

1(8) 원소를 삽입한 후의 이진검색트리:

9(5)

7(1)

6(3)

5(4)

4(2)

3(6)

2(0)

1(8)

0(7)

8(9) 원소를 삽입한 후의 이진검색트리:

9(5)

8(9)

7(1)

6(3)

5(4)

4(2)

3(6)

2(0)

1(8)

0(7)

□ 출력 화면 [2] : 중위 탐색 단계

<< Inorder Traversal >>

0 (7)

1 (8)

2 (0)

3 (6)

4 (2)

5 (4)

6 (3)

7 (1)

8 (9)

9 (5)

출력 화면 [3] : 삭제 단계

<< 삭제 과정에서의 이진검색트리의 변화 >>

Key 값이 2 인 원소를 삭제한 후의 이진검색트리:

```

      9( 5)
     /  \
    7( 1) 8( 9)
     \  /
    6( 3) 5( 4)
     \ /
    4( 2) 3( 6)
     \ /
    1( 8) 0( 7)
  
```

Key 값이 7 인 원소를 삭제한 후의 이진검색트리:

```

      9( 5)
     /  \
    6( 3) 8( 9)
     \  /
    4( 2) 5( 4)
     \ /
    3( 6) 1( 8)
     \ /
    0( 7)
  
```

Key 값이 4 인 원소를 삭제한 후의 이진검색트리:

```

      9( 5)
     /  \
    6( 3) 8( 9)
     \  /
    3( 6) 5( 4)
     \ /
    1( 8) 0( 7)
  
```

Key 값이 6 인 원소를 삭제한 후의 이진검색트리:

```

      9( 5)
     /  \
    5( 4) 8( 9)
     \  /
    3( 6) 1( 8)
     \ /
    0( 7)
  
```

Key 값이 5 인 원소를 삭제한 후의 이진검색트리:

```

      9( 5)
     /  \
    3( 6) 8( 9)
     \  /
    1( 8) 0( 7)
  
```

Key 값이 9 인 원소를 삭제한 후의 이진검색트리:

```

      8( 9)
     /  \
    3( 6) 1( 8)
     \  /
    0( 7)
  
```

Key 값이 3 인 원소를 삭제한 후의 이진검색트리:

```

      8( 9)
     /  \
    1( 8) 0( 7)
  
```

Key 값이 0 인 원소를 삭제한 후의 이진검색트리:

```

      8( 9)
     /  \
    1( 8)
  
```

Key 값이 1 인 원소를 삭제한 후의 이진검색트리:

```

      8( 9)
  
```

Key 값이 8 인 원소를 삭제한 후의 이진검색트리:

손쉬운 해결 방법과 문제점

□ 중위 탐색의 손쉬운 해결책

```
public class DictionaryByBinarySearchTree<...> extends Dictionary<...>
```

```
{
```

```
.....
```

캡슐화를 무력화 시키고 있다

```
private void visit(DictionaryElement<...> anElement) {
```

```
    AppView appView = new AppView() ;
```

```
    appView.outputLine (
```

```
        String.format ("%4d (%2d)", anElement.key(), anElement.object() );
```

```
    }
```

```
private void inorderRecursively (BinaryNode<...> aRootOfSubTree) {
```

```
    if ( aRootOfSubTree != null ) {
```

```
        this.inorderRecursively (aRootOfSubTree.left()) ;
```

```
        this.visit (aRootOfSubTree.element()) ;
```

```
        this.inorderRecursively (aRootOfSubTree.right()) ;
```

```
    }
```

```
}
```

```
public void inOrder() {
```

```
    this.inorderRecursively (this.root()) ;
```

```
}
```

```
} // End of class "DictionaryByBinarySearchTree"
```

□ 문제점은?

■ 설계의 근본적 관점에서:

Model-View-Controller 의 개념이 깨졌다!

- Model ("DictionaryByBinarySearchTree")이 View ("AppView")의 기능을 사용하고 있다.

■ 실용적 관점에서:

visit() 에서 해야 할 일이 나중에 바뀐다면?

- 그때마다 visit() 를 수정하여 사용?

Call Back



□ 이진트리의 Traversal 을 구현할 때 생각해야 할 점

- 이진트리의 탐색은 구현에 의존적이다.
 - 그러므로, 탐색 자체는 이진 트리 내부에 구현되어야 할 일이다.
- 그러나,

특정 노드를 방문했을 때 해야 할 일은, 이진트리의 사용자가 정의할 수 있어야 한다. 즉, 이진 트리를 사용하는 사람의 용도에 맞게 표현되어야 한다. 그렇다는 것은, 방문했을 때 해야 할 일을 이진 트리를 구현하는 class 내부에 정의할 수는 없다.

 - 이진 트리 class 내부에 구현하는 방법으로 해결하는 것은, Model-View-Controller 의 개념을 깨뜨리게 될 위험성이 높다.
 - ◆ 방문 시에 해야 할 일이 출력할 일이라면.....
 - 또한 고정시켜 버린 방문(visit) 행위는, "DictionaryByBinarySearchTree" 객체를 사용하는 또 다른 사용자가 자신의 목적에 맞는 용도로 사용할 수 없게 만든다. 아니면 각 사용자는 자신 만의 방문 행위를 포함하는 자신 만의 목적에 맞는 "DictionaryByBinarySearchTree" class 를 정의하여 사용해야 할 것이다.

□ 문제의 요약 [1]

■ 주어진 상황:

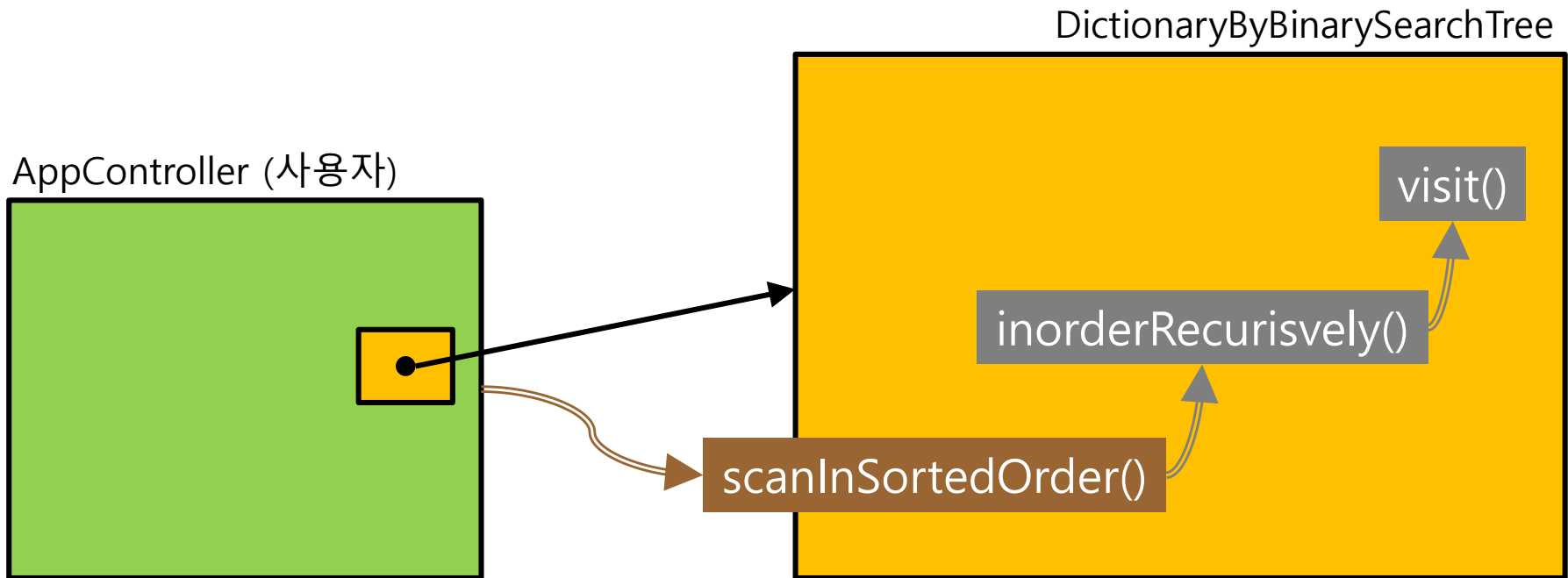
- 우리가 사용할 이진트리를 구현하는 class는 "DictionaryByBinarySearchTree" 이다.
- 이진트리 탐색은 inorder traverse (중위 탐색) 을 실행한다:

```
public class DictionaryByBinarySearchTree<...> extends Dictionary<...>
{
    .....

    private void inorderRecursively (BinaryNode<...> aRootOfSubTree) {
        if ( aRootOfSubTree != null ) {
            this.inorderRecursively (aRootOfSubTree.left()) ;
            this.visit() ;
            this.inorderRecursively (aRootOfSubTree.right()) ;
        }
    }
    public void scanInSortedOrder() {
        this.inorderRecursively (this.root());
    }
} // End of class "DictionaryByBinarySearchTree"
```

□ 문제의 요약 [2]

- "visit()"는 class "DictionaryByBinarySearchTree"의 private method.
- ApplicationController로서는 "visit()"의 내용을 자신의 목적에 맞게 구현할 수 있을까?



□ 문제의 요약 [3]

■ 문제점:

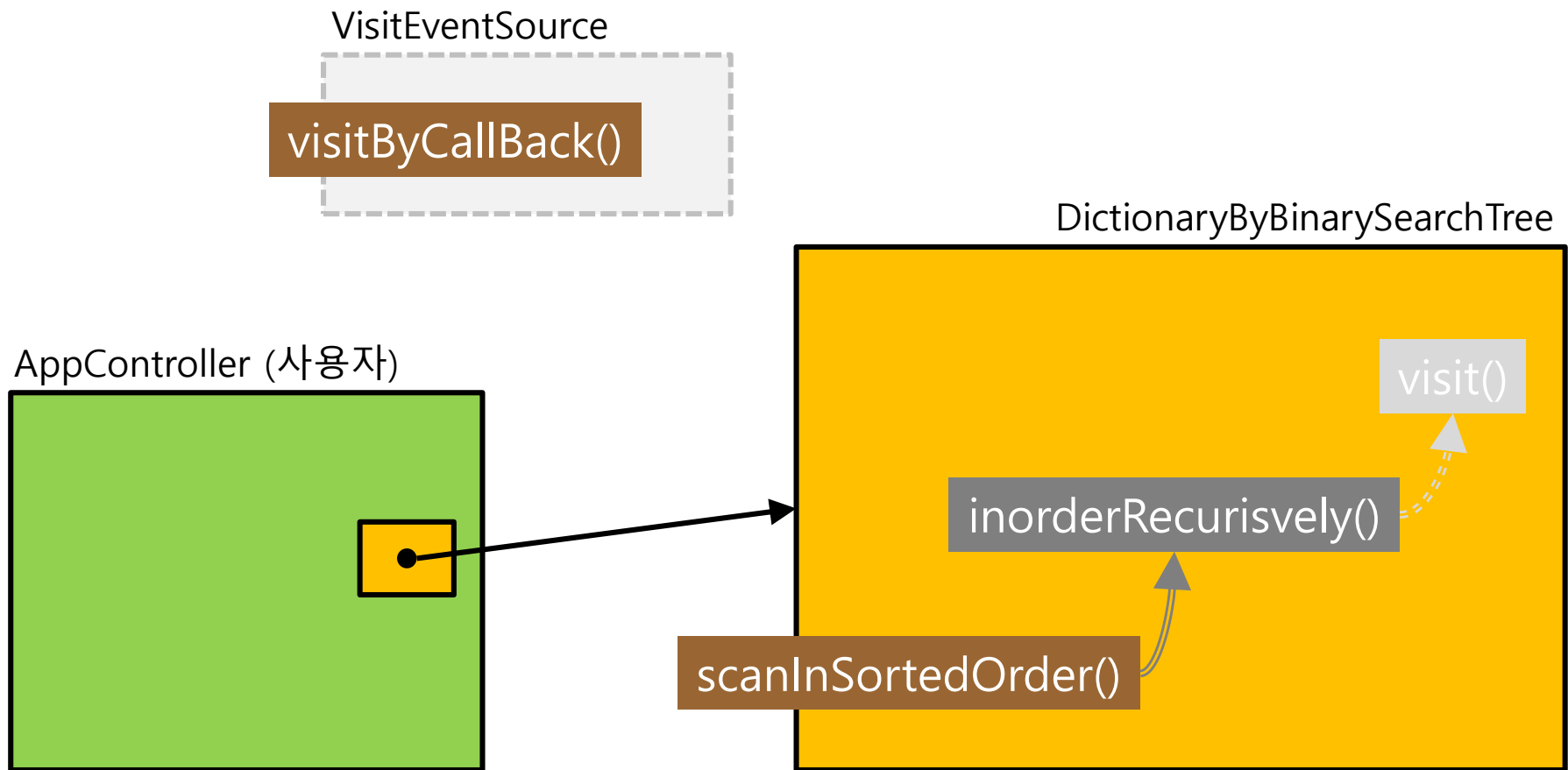
- "inorderRecursively()" 내부에서 call 하는 함수 "visit()" 는, **형식적인 관점에서** 자신의 class "DictionaryByBinarySearchTree" 안에 정의하는 것이 보통의 방법일 것이다.
- 그러나 **실제적인 관점에서는** 함수 "visit()"의 내용은, class "DictionaryByBinarySearchTree" 객체의 **사용자가 자신의 목적에 맞게** 정의할 수 밖에 없다.
- 또한 사용자 마다 사용 목적이 다를 수 있으므로 "visit()" 의 내용이 달라질 것이다.
- 그렇다면, inorder traverse 의 사용자에게, 자신의 원하는 바를 class "DictionaryByBinarySearchTree" 내부의 함수 visit() 를 직접 구현할 수 있게 허락해야 할까? 허락하면 어떤 문제가 있을까? 허락하지 않는다면 해결책은?

■ 해결책:

- "visit()"의 구현은 사용자가, 실행은 "DictionaryByBinarySearchTree" 객체가!!
- 어떻게?

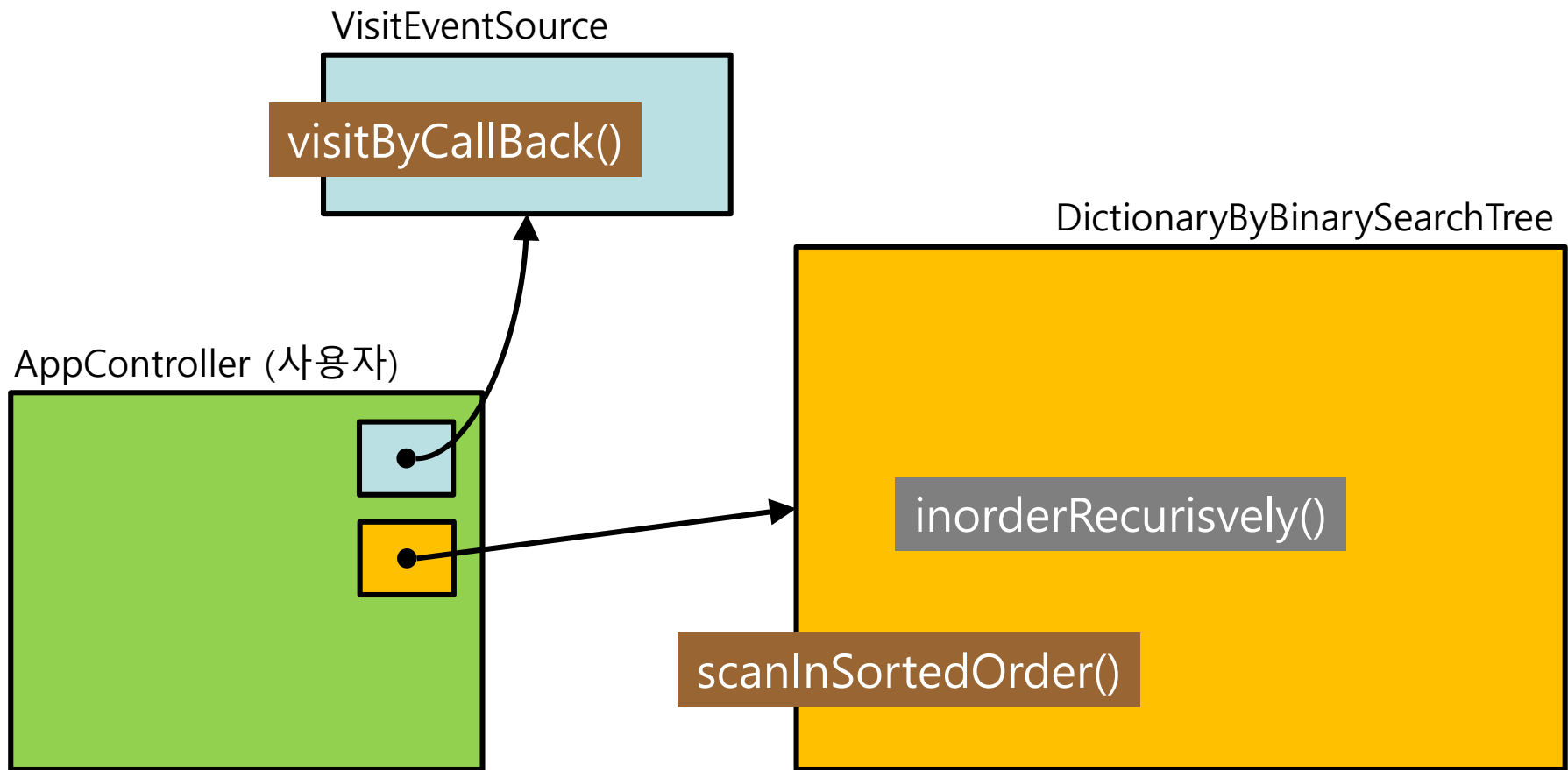
□ 해결책은? [1]

- "DictionaryByBinarySearchTree" 는 "VisitEventSource"를 **Java Interface** 로 정의하여 사용자에게 제공한다. 즉 함수 "**visitByCallBack()**" 의 사용법만 정의하게 된다. 결국 "visitByCallBack()" 이 "visit()"의 역할을 하게 된다.



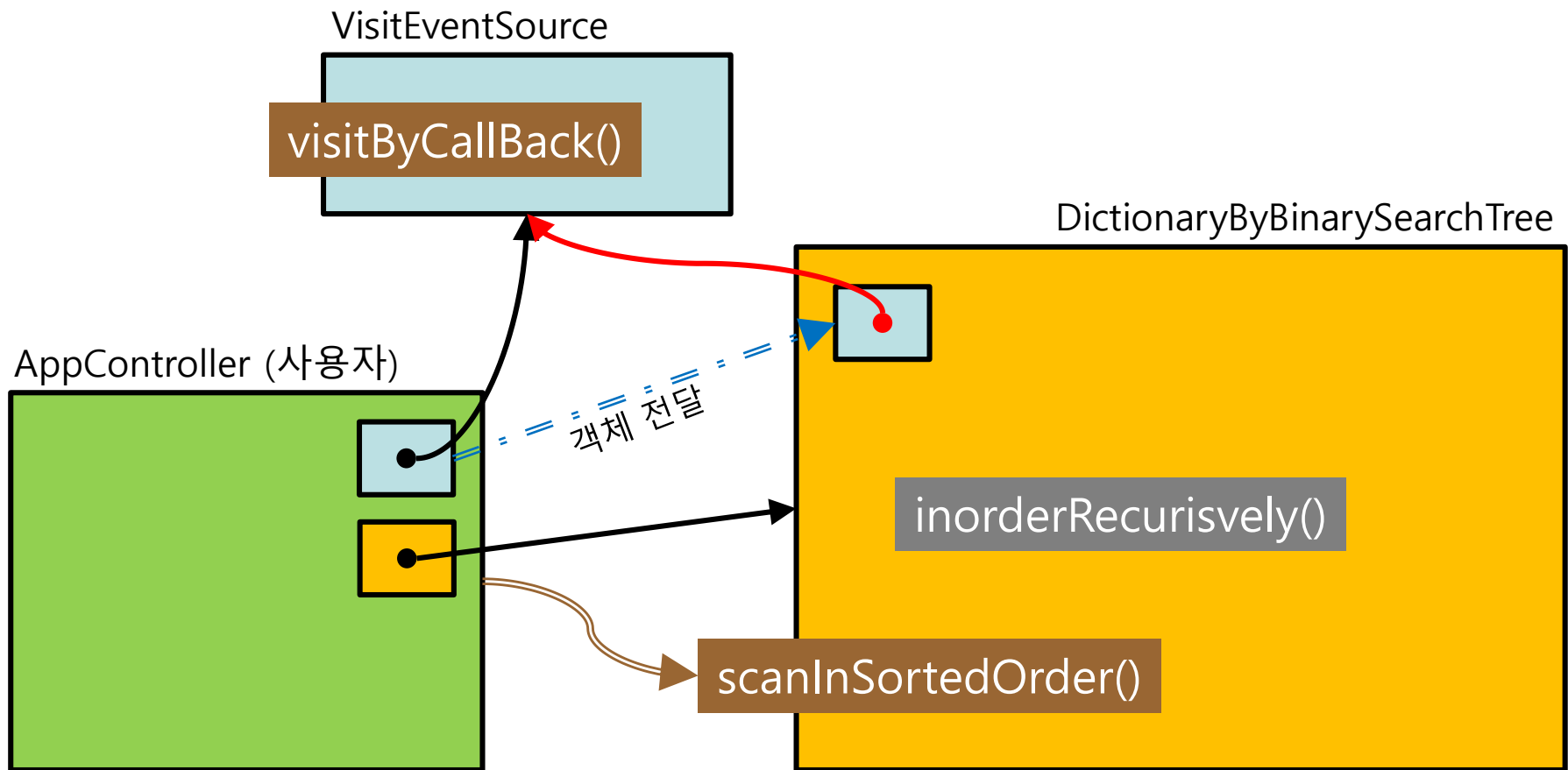
□ 해결책은? [2]

- 사용자는 인터페이스 "VisitEventSource" 를 구현해 놓는다.
- 사용하는 시점에 사용자는 "VisitEventSource" 객체를 생성한다.



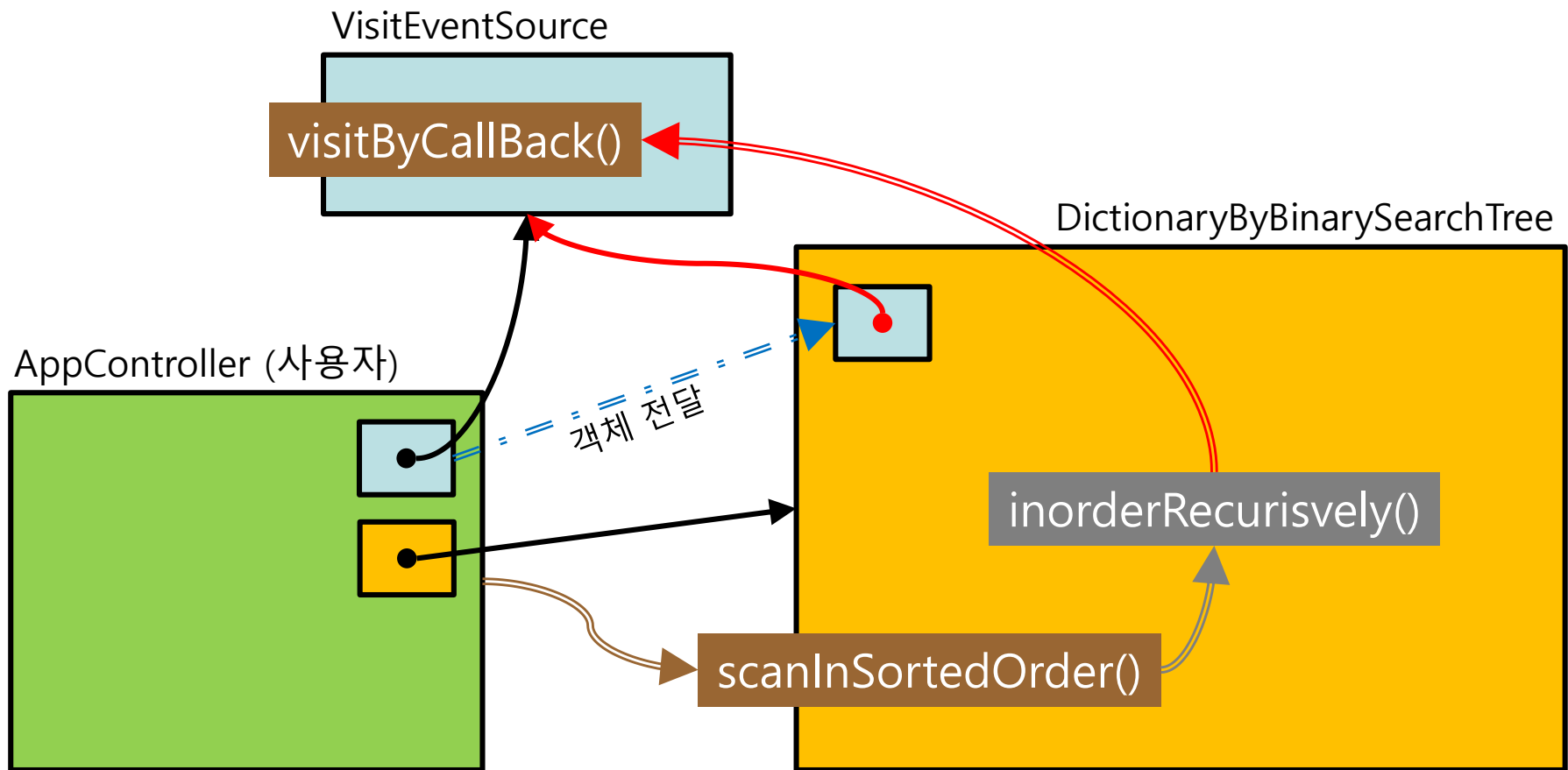
□ 해결책은? [3]

- 사용자는 "VisitEventSource" 객체를 "DictionaryByBinarySearchTree" 객체로 전달한다.
- 사용자는 "DictionaryByBinarySearchTree" 객체에게 "scanInSortedOrder()"를 실행하게 한다.



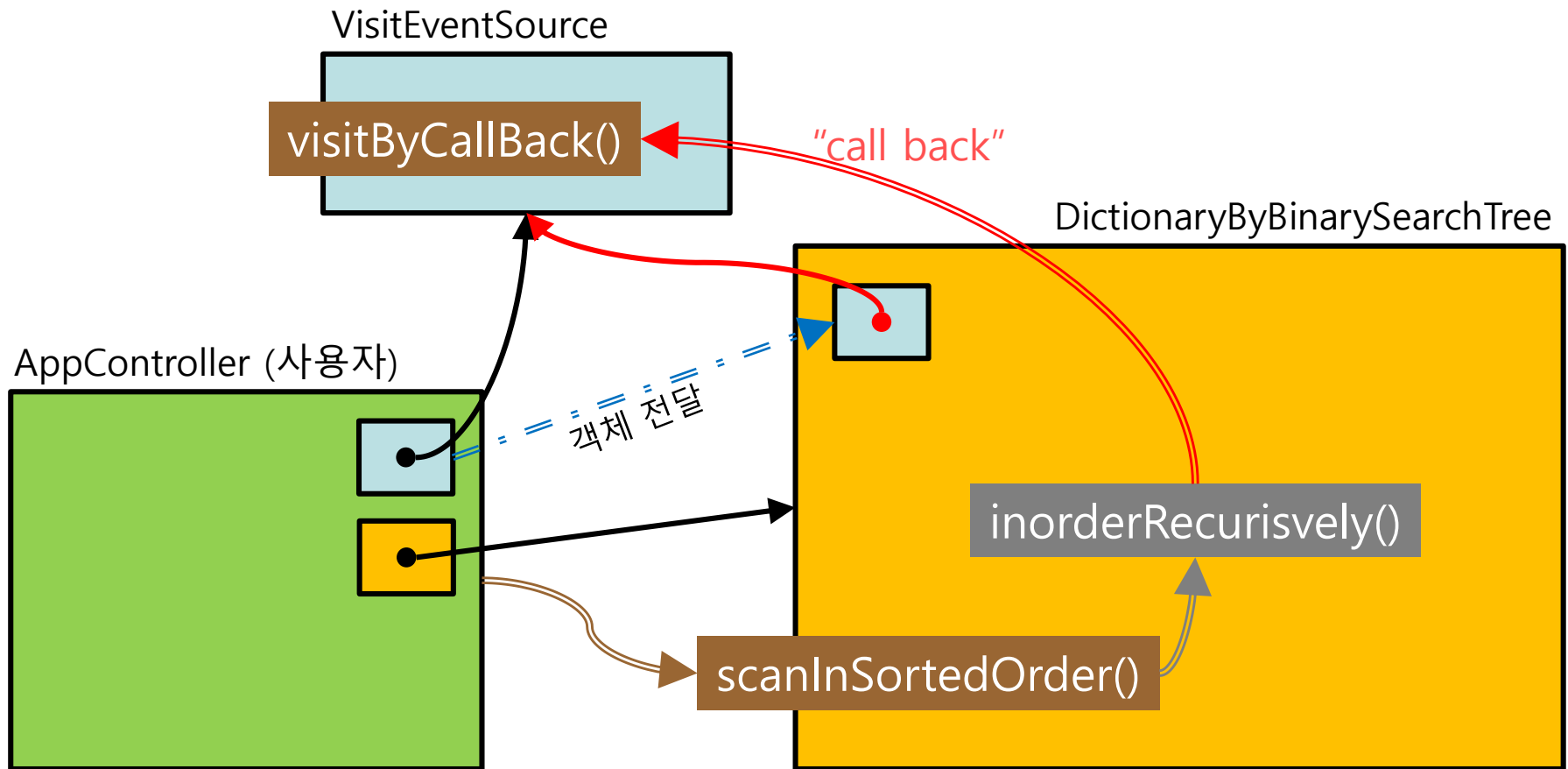
□ 해결책은? [4]

- "DictionaryByBinarySearchTree" 객체는 전달받은 "VisitEventSource" 객체에게 "visitByCallback()"을 실행하게 한다.
- 이로서, <visit 의 행위는 사용자가 정의하고, 실행은 "DictionaryByBinarySearchTree"가 한다> 는 우리의 목적이 달성될 수 있다.



Call Back?

- AppController 객체는 VisitEventSource 객체를 생성하여 소유하고 있다. 그리고 사전 (DictionaryByBinarySearchTree) 객체에게 "inorder()" 라는 일을 시키려고 call 하고 있다. 사전 객체는 자신의 사용자인 AppController 객체가 소유한, 그리고 지금 자신에게 전달한 VisitEventSource 객체에게 일을 시킨다. 이 행위는 자신을 call 한 객체 쪽으로 되돌아 call 을 하는, 즉 **call back** 하는 셈이 된다.



Interface "VisitEventSourceForTreeTraversal"

□ Interface VisitEventSource

- Class "DictionaryByBinarySearchTree"의 트리 탐색 (traverse)을 실행하는 함수에서 (예를 들어 inorder()), 노드를 방문(visit) 할 때 마다 실행할 Callback 함수 사용법 (함수의 이름과 매개변수)를 정의하기 위한 Interface.

```
public interface VisitEventSource<Key,Obj> {
    public void visitByCallback (DictionaryElement<Key,Obj> anElement, int aLevel) ;
}
```

- Class "DictionaryByBinarySearchTree"와 사용자 class 는 Callback 을 위한 interface "VisitEventSource"를 공유한다.
- Callback 함수 "visitByCallback()" 의 매개변수:
 - ◆ 사용자는 트리를 탐색하는 동안 노드를 방문할 때 마다, 방문 노드의 원소 (anElement)와 그 노드의 트리 레벨 (aLevel) 정보를 Callback 함수를 통해서 전달받게 된다.
 - ◆ 이 두 정보를 사용하여 사용자는 자신의 목적에 맞는 일을 구현할 수 있다.

Class "DictionaryByBinarySearchTree" 의 수정

DictionaryByBinarySearchTree의 수정 [1]

```

public class DictionaryByBinarySearchTree <Key extends Key, Obj> extends Dictionary<Key,Obj> {
    ....

    private VisitEventForDictionaryByBinarySearchTree _visitEvent ;
        // 전달받은 Visit event 객체를 저장할 곳

    public VisitEventForDictionaryByBinarySearchTree visitEvent () {
        return this._visitEvent ;
    }
    public void setVisitEvent (VisitEventForDictionaryByBinarySearchTree newVisitEvent) {
        this._visitEvent = newVisitEvent ;
    }

    .....

    private void inorderRecursively (
        BinaryNode<DictionaryElement<Key,Obj>> aRootOfSubtree,
        int aLevel )
    {
        if ( aRootOfSubtree != null ) {
            this.inorderRecursively (aRootOfSubtree.left(), aLevel+1) ;
            this.visitEvent().visitByCallBack (aRootOfSubtree.element(), aLevel) ;
            this.inorderRecursively (aRootOfSubtree.right(), aLevel+1) ;
        }
    }

    public void scanInSortedOrder () {
        this.inorderRecursively (this.root(), 1) ;
    }
}

```


□ DictionaryByBinarySearchTree의 수정 [2]

```
public class DictionaryByBinarySearchTree <Key extends Key, Obj> extends Dictionary<Key,Obj> {
    ....

    private void inorderRecursively (...) {...}
    public void inorder () {...}

    private void reverseOfInorderRecursively (
        BinaryNode<DictionaryElement<Key,Obj> > aRootOfSubtree, int aLevel)
    {
        if ( aRootOfSubtree != null ) {
            this.reverseOfInorderRecursively (aRootOfSubtree.right(), aLevel+1) ;
            this.visitEvent().visitByCallBack (aRootOfSubtree.element(), aLevel) ;
            this.reverseOfInorderRecursively (aRootOfSubtree.left(), aLevel+1) ;
        }
    }
    public void scanReverseOfSortedOrder () {
        this.reverseOfInorderRecursively (this.root(), 1) ;
    }
}
```

Class "AppController" 의 수정

□ ApplicationController: Call Back 관련 수정[1]

```
public class ApplicationController implements VisitEventSourceForTreeTraversal<Integer, Integer>
{
```

```
// Constants
private static final int
    DEFAULT_DATA_SIZE = 10 ;
```

```
// Private instance variables
private AppView _appView ;
private DictionaryByBinarySearchTree<Integer,Integer> _dictionary ;
private int[] _list ;
```

```
...
```

```
public void run() {
    this._list = DataGenerator.randomList(DEFAULT_DATA_SIZE) ;

    this._appView.outputLine(...) ;
    this._dictionary = new DictionaryByBinarySearchTree<Integer, Integer>() ;
```

```
    this._dictionary.setVisitEventSource (this) ;
```

```
    this.addToBinarySearchTreeAndShowShape() ;
    this.showInorderOfBinarySearchTree() ;
    this.removeFromBinarySearchTreeAndShowShape() ;
}
```

별도의 event class 를 정의하여, event source 객체를 생성하는 방법을 사용할 수도 있다. 그러나 여기서는 그렇게 하지 않고, ApplicationController 자신을 event source 객체로 하는 방법을 사용한다.

Dictionary 객체에게 Visit event의 source 가 어느 객체인지를 알려 준다. 여기서는 ApplicationController 객체 자신이므로 **this** 를 전달한다.

□ ApplicationController: Call Back 관련 수정[2] ^{사전}

```
public class ApplicationController {  
  
    .....  
  
    public void run() {.....}  
  
    @Override  
    public void visitByCallBack (DictionaryElement<Integer, Integer> anElement, int aLevel) {  
        // TODO Auto-generated method stub  
        ..... // 여기를 채우시오  
    }  
  
    @Override  
    public void visitInReverseOrder (DictionaryElement<Integer, Integer> anElement, int aLevel) {  
        // TODO Auto-generated method stub  
        ..... // 여기를 채우시오  
    }  
}
```

요약

보고서 작성과 제출



□ PDF 보고서 작성 방법

■ 겉장

- 제목: 자료구조 실습 보고서
- [제14주] 사전의 성능 분석 / Call Back 의 구현
- 제출일
- 학번/이름

■ 내용

1. 프로그램 설명서
 1. 주요 알고리즘 /자료구조 /기타
 2. 함수 설명서
 3. 종합 설명서
2. 프로그램 장단점 분석
3. 실행 결과 분석
 1. 입력과 출력 (화면 capture 시킬 것)
 2. 결과 분석
4. 소스코드

□ 과제 제출

■ 파일 이름 작명 방법

● DS1_14_학번_이름.zip

● 폴더의 구성

◆ DS1_14_학번_이름

■ 프로그램

– 프로젝트 폴더 / 소스

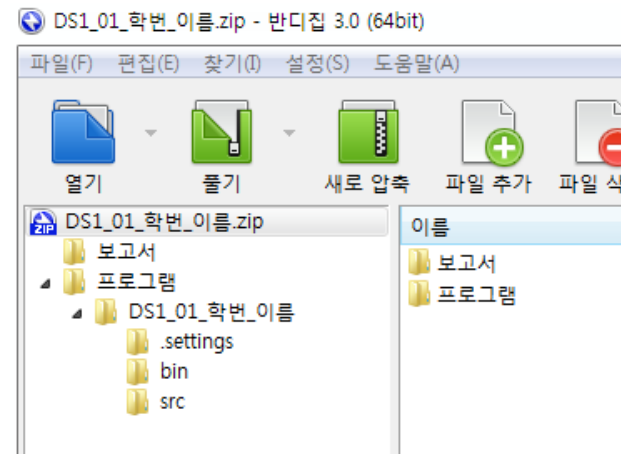
– 메인 클래스 이름 : DS1_14_학번_이름.java

■ 보고서

– 이곳에 보고서 문서 파일을 저장한다.

– 입력과 실행 결과는 화면 image로 문서에 포함시킨다.

– 문서는 pdf 파일로 만들어 제출한다.



[제 14 주] 끝

한 학기 동안 수고했습니다!

훌륭한 IT 전문가가 되기를
기원합니다!



